Farmer Helper - AI for Farmers & Agriculture
Complete System Architecture & Documentation

Project Overview

Farmer Helper is an AI-powered platform that leverages machine learning, deep learning, and cloud integration to support farmers in daily agricultural decisions. It offers crop recommendations, plant disease and weed detection, weather forecasts, market insights, and an AI chatbot for instant assistance. The system aims to make farming smarter, more profitable, and less dependent on guesswork.

Sentimental Problem Statement

Across India, millions of farmers face challenges like unpredictable weather, pest infestations, poor crop yield, and lack of access to real-time agricultural data. Many rely on traditional methods or local advice that may not reflect scientific accuracy. Farmer Helper bridges this gap by bringing AI, data science, and real-time analytics directly to the farmer's fingertips. It transforms raw agricultural data into meaningful insights that can save time, money, and effort.

System Architecture Overview

Farmer Helper uses a multi-layered microservice architecture for high scalability and modularity.

Frontend (React + Vite + Tailwind CSS)
↓
Backend (Node.js + Express)
↓
ML Services (Python + Flask/FastAPI)
↓
Machine Learning Models (Crop, Disease, Weed)
↓
External APIs (Weather, Market Data)
↓
Dialogflow Chatbot (AI Assistant)

Each layer communicates via RESTful APIs to maintain loose coupling, scalability, and independent development.

A) FRONTEND (React + Vite + Tailwind CSS)

Folder Structure

```
frontend/
├── public/              # Static assets
├── src/
│   ├── assets/          # Images, icons, and reusable visuals
│   ├── Dashboard/       # Core modules (Crop, Disease, Market, Weather)
│   ├── LoginPage/       # Authentication components
│   ├── landing_page/    # Home and marketing UI
│   ├── hooks/           # Custom React hooks
│   ├── App.jsx          # Root app router and layout manager
│   ├── backend.js       # Axios setup and backend API configuration
│   ├── RequireAuth.jsx  # Route protection for private pages
│   └── unique_values.json # Predefined datasets for dropdowns
```

Component Explanation

Dashboard/

- CropSelection.jsx – Collects soil and environmental parameters; fetches crop recommendations.
- DiseaseDetection.jsx – Allows image uploads; displays detected plant disease or weed result.
- MarketPrice.jsx – Displays live commodity prices from government APIs.
- Weather.jsx – Fetches real-time weather forecasts using OpenWeatherMap API.
- Overview.jsx – Dashboard summary showing recent farm insights.
- Profile.jsx – Handles user data (farm location, soil type, etc.).
- Sidebar.jsx – Navigation bar linking all dashboard features.
- Dashboard.jsx – Integrates all components into a cohesive layout.

landing_page/

- Hero.jsx – Showcases the platform's mission and benefits.
- Features.jsx – Highlights AI-based features (Crop Recommendation, Disease Detection, Chatbot, etc.).
- Feedback.jsx – Displays testimonials or user feedback.
- Stats.jsx – Dynamic statistics (active farmers, datasets, predictions made)

.

LoginPage/

- Login.jsx – Authenticates users via backend /api/auth/login.
- SignUp.jsx – Registers new users through /api/auth/register.
- RequireAuth.jsx – Protects sensitive routes using JWT authentication.

hooks/

- useReveal.js – Provides animation triggers for component appearance.

Utility Files

- backend.js – Centralized Axios setup for API calls.
- unique_values.json – Contains static options for dropdown menus (crop types, soil, regions).

B) BACKEND (Node.js + Express)

Purpose

Acts as the API Gateway and integration layer, managing communication between the frontend, ML services, database, and external APIs.

Folder Structure

```
backend/
├── config/          # Database setup (MySQL/MongoDB)
├── controllers/     # Business logic for all modules
├── middlewares/     # Authentication and validation
├── models/          # Database schemas
├── routes/          # API endpoints mapping
├── uploads/         # Temporary file storage (e.g., leaf images)
└── server.js        # Express entry point
```

Controllers Breakdown

- authController.js – Handles login, signup, and JWT token creation.
- cropController.js – Sends user soil data to ML microservice for prediction.
- diseaseController.js – Forwards uploaded images to ML inference API (weed/disease detection).
- marketController.js – Fetches live crop price data from Agmarknet APIs.
- userController.js – Manages user profile CRUD operations.
- weatherController.js – Retrieves real-time weather data from OpenWeatherMap.

Middleware

- authMiddleware.js – Verifies JWT tokens for protected endpoints.

Models

- User.js – Stores name, email, hashed password, farm location, and preferences.

server.js

- Initializes Express app.
- Registers routes, CORS, and error handling.
- Connects backend with Python ML microservices via HTTP.

C) ML SERVICES (Python + Flask/FastAPI)

Purpose

Hosts all machine learning and deep learning models independently as microservices. Each module can be scaled or updated without touching the Node.js backend.

Folder Structure

```
ml_services/
├── models/       # Stored serialized models (.pkl / .pt)
├── routes/       # API endpoints for each ML task
├── services/     # Model logic (data preprocessing, inference)
├── uploads/      # Temporary image storage
├── app.py        # Flask main entry point
├── requirements.txt
└── dialogflow-key.json
```

Models Used

- crop_recommendation_model.pkl – Predicts the best crop using N, P, K, temperature, humidity, pH, rainfall.
- disease_model.pt (YOLOv8) – Detects plant diseases from uploaded leaf images.
- weed_detection_model.pt (YOLOv8) – Identifies weeds among crops.
- label_encoder.pkl – Maps model output classes to readable crop/disease names.

Routes

- crop_routes.py – /predict_crop endpoint for crop recommendation.
- disease_routes.py – /detect_disease endpoint for disease/weed detection.
- dialogflow_bp.py – Chatbot communication handler for AI conversation.

Services

- crop_service.py – Handles preprocessing, loads model, predicts and returns JSON.
- disease_service.py – Runs YOLOv8 inference and returns detection results.

Chatbot Integration (Dialogflow)

- Powered by Google Dialogflow, offering multilingual support (Marathi, Hindi, English).
- Integrated via Node.js backend using the Dialogflow API and Service Account Key (dialogflow-key.json).
- Responds to queries like:
    - "Which crop is best for my soil?"
    - "What disease is on my tomato plant?"
    - "Show me market price of wheat."
    - "What will be the weather tomorrow?"
- Supports both voice and text input.

Features

- Natural Language Understanding (NLU) for complex queries.
- Contextual memory for follow-up questions.
- Can connect live with ML models for on-demand predictions.

Live APIs Integration

- Weather API: OpenWeatherMap API to fetch forecasts and temperature/humidity data.
- Market Price API: Agmarknet or government APIs to display real-time commodity prices.
- Geolocation API: Determines user farm location for region-specific insights.

Data Flow

1. User interacts with frontend.
2. Request goes to backend → relevant controller.
3. Backend forwards data/image → ML microservice.
4. ML model returns prediction → backend formats response.
5. Frontend displays insights or passes to chatbot for explanation.

Security Measures

- JWT Authentication for secure access.
- CORS Enabled for cross-origin safety.
- Multer Middleware for safe image uploads.
- Hashed Passwords (bcrypt) to ensure credential safety.

Key Advantages

- Modular and scalable architecture.
- AI-driven precision agriculture.
- Independent deployment for each microservice.
- Extensible to add IoT or drone data integration in the future.

Future Enhancements

- Integration with IoT soil sensors.
- Advanced NLP-based chatbot for deeper agricultural Q&A.
- Predictive yield estimation model.
- Mobile app version for offline access.

Summary

Farmer Helper bridges the gap between traditional farming and modern agriculture through AI innovation. With crop intelligence, disease detection, market awareness, and real-time chatbot support — it’s not just an app, it’s a digital partner for every farmer’s success journey.